PRPrompt: Prototype-Routed Graph Prompting for Few-Shot Node Classification

Abstract

We introduce PRPrompt, a graph prompting method that uses support-set prototypes to route trainable prompts to individual nodes. With only a few labeled nodes per class, adaptation must determine both what a prompt should encode and where its contribution should be applied. PRPrompt constructs class references from aggregated node attributes, refines them using unlabeled graph features while retaining support anchors, and converts node–prototype associations into a fixed routing matrix. The resulting combinations of class prompt tokens are injected into a frozen graph convolutional encoder, followed by prototype classification. We evaluate each 1-, 3-, and 5-shot setting separately on Cora, CiteSeer, and PubMed using matched support draws and training seeds. In the 5-shot setting, PRPrompt achieves accuracies of 73.02%, 66.17%, and 70.98%, respectively; replacing a shared prompt with prototype routing improves accuracy by 36.94, 40.45, and 3.02 percentage points in the corresponding conditions. These paired results support the value of prototype-guided prompt allocation within the evaluated transductive setting.

1 Introduction

Pretrained graph neural networks offer a starting point for learning with scarce node labels. Graph prompting adapts these representations through a small set of task parameters, using feature modifications, prompt structures, or task-specific readout [2, 9, 13, 14]. Beyond choosing an injection point, an adaptation method must decide how a prompt's contribution is distributed across nodes. This allocation matters because nodes in the same graph can play different roles in the downstream classification task.

A shared prompt expresses a global adjustment. More flexible methods learn input-dependent mixtures, assign prompts to subgraphs, or select nodes through a policy [2, 7, 21]. These approaches establish that node-dependent allocation is not itself a new idea. Our question is more specific: *can the few labeled examples already available for a task define useful routing coefficients, so that optimization can focus on prompt content?* Support examples provide class references that can be constructed before adaptation and used without training an additional routing network.

We propose PRPrompt, which uses support-anchored prototypes as coordinates for prompt allocation. Nodes with similar class associations receive related combinations of class prompt tokens. The routing representation comes from aggregated graph attributes; its prototypes are initialized by labeled support nodes and refined with unlabeled features while retaining the support anchors. These associations yield a fixed routing matrix that mixes trainable tokens inside a frozen GCN. Only the prompt tokens are optimized, and prediction uses a prototype readout. Figure 1 summarizes this separation between fixed allocation and learned content.

Our evaluation asks two questions. First, does the complete adaptation improve upon frozen pretraining, full fine-tuning, and the evaluated prompt operators under a common protocol? Second, does prototype routing add value over a shared prompt when the injection point and training recipe are held fixed? We answer them using existing matched runs on Cora, CiteSeer, and PubMed at 1, 3, and 5 shots. Every result remains specific to its dataset, shot count, and checkpoint. The gain over frozen pretraining is much smaller on PubMed than on Cora or CiteSeer, making this separation essential to the interpretation.

The paper makes three contributions:

- A graph prompting construction that uses support-anchored class relationships to fix node-specific allocation before prompt optimization.
- A hidden-layer implementation with $C \times 128$ trainable prompt parameters and no learned routing network.
- Matched adaptation comparisons and shared-prompt ablations, with seed variability and paired uncertainty reported separately for each evaluated condition.

The contribution is the use of prototype relationships to organize hidden-layer prompts; it does not treat prototypes or node-dependent prompting as new primitives.

2 Related Work

Graph representations and pretraining. Graph convolution, neighborhood sampling, and graph attention provide complementary ways to construct node representations [3, 6, 15]. Self-supervised objectives include local–global information maximization [16], augmentation-based contrastive learning [17], and masked reconstruction [4]. Pretraining objectives can affect downstream transfer, and combining node- and graph-level objectives has been studied explicitly [5]. Our study fixes the pretrained checkpoint on each graph and examines downstream adaptation,

rather than proposing a new pretraining objective.

Aligning pretraining with downstream tasks. GPPT and GraphPrompt connect pretraining to downstream prediction through task reformulation and prompting [9, 13]. All in One addresses multiple task levels with prompt tokens, structures, and insertion rules [14]; MultiGPrompt combines multiple pretext tasks and downstream prompts [19]. HGPrompt and HetGPT extend prompting to heterogeneous graph settings [10, 18]. These methods address task or graph-type alignment. We focus on a narrower design question within few-shot node classification: how task-class references should allocate prompts across nodes.

Adaptive prompt allocation. GPF-plus learns input-dependent prompt combinations [2], SUPT assigns prompts at subgraph level [7], and RELIEF learns node selection together with prompt content [21]. ProNoG studies conditional node-specific prompts for non-homophilic graphs [20], while DAGPrompT addresses distribution-aware adaptation [1]. Attribute-guided dynamic prompting has also been studied for robustness to graph perturbations [8]. In contrast to learning an allocation function, PRPrompt computes support-conditioned prototype relationships once and keeps the resulting route fixed. These works delimit the design space; they are not all experimental baselines in our controlled comparison.

Class prototypes and evaluation. Prototypical Networks use support-set class representatives for prediction [12], and semi-supervised extensions incorporate unlabeled examples into prototype construction [11]. We use these established ideas to obtain task references for routing. Routing prototypes live in aggregated attribute space; prediction prototypes come from the adapted encoder output. Following the motivation for comparable graph-prompt evaluation in ProG [22], we state the shared backbone and fitting protocol explicitly. The reported comparisons characterize these implementations, rather than each method's best published configuration.

3 Problem Setting

Let $G = (V, E, X)$ be an attributed graph with n nodes, features $X \in \mathbb{R}^{n\times d}$, and C task classes. For each class c, a support set S_c contains k labeled nodes, with $k \in \{1, 3, 5\}$. Let $S = \bigcup_c S_c$ and let T be a disjoint query set. The full graph topology and all node features are visible; only labels in S are provided to fitting and hyperparameter selection. Query labels are used for scoring after predictions are sealed.

The encoder is a pretrained two-layer GCN [6] with hidden and output dimensions of 128. Its parameters are frozen for PRPrompt. The task is transductive, same-graph classification over known classes: query features participate in graph aggregation, prototype refinement, and global centering. It is not an inductive unseen-class evaluation. Throughout the paper, a *condition* fixes the dataset, shot count, method, and pretrained checkpoint.

4 Prototype-Routed Prompting

Overview. The method separates allocation from content. First, support-anchored prototypes define class relationships in aggregated attribute space. Second, a fixed routing matrix turns those relationships into node-specific combinations of prompts. Third, support supervision learns the prompt tokens inside the frozen encoder. The following subsections follow this computation order.

4.1 Support-anchored class references

Let $\widehat{A} = \widetilde{D}^{-1/2}(A + I)\widetilde{D}^{-1/2}$ denote the normalized adjacency with self-loops. We form a routing representation from two-hop aggregated attributes:

$$\begin{aligned} X^{(2)} &= \widehat{A}^2 X, \\ R &= \mathrm{rownorm}\Big(X^{(2)} - \mathbf{1}\,\mathrm{mean}_{v\in V}\, X_v^{(2)}\Big). \end{aligned} \tag{1}$$

Here rownorm applies Euclidean normalization to each row. Routing therefore uses graph attributes and structure, rather than a separate learned network. Initial class references are

$$\mu_c^{(0)} = \frac{1}{k}\sum_{i\in S_c} R_i. \tag{2}$$

To incorporate the unlabeled graph while retaining a labeled reference, let $U_V = V \setminus S$. Starting from $\mu^{(0)}$, we use $i \in U_V$ and $t = 0, 1, 2$ for three fixed refinement iterations:

$$a_{ic}^{(t)} = \frac{\exp(R_i^\top \mathrm{norm}\Big(\mu_c^{(t)}\Big)/0.05)}{\sum_{c'}\exp(R_i^\top \mathrm{norm}\Big(\mu_{c'}^{(t)}\Big)/0.05)}, \tag{3}$$

$$\mu_c^{(t+1)} = 0.5\mu_c^{(0)} + 0.5\frac{\sum_{i\in U_V} a_{ic}^{(t)} R_i}{\max(\sum_{i\in U_V} a_{ic}^{(t)}, 10^{-8})}. \tag{4}$$

The anchor is always the original support prototype, not the previous iterate. This is a fixed prototype construction; we do not claim an expectation-maximization objective or a convergence guarantee. No labels from U_V enter these computations.

4.2 From class relationships to a fixed routing matrix

Using the refined prototypes, we compute node–class associations at temperature $\tau_r = 0.1$:

$$\begin{aligned} Q_{ic} &= \mathrm{softmax}_c\Big(R_i^\top \mathrm{norm}\Big(\mu_c^{(3)}\Big)/\tau_r\Big), \\ B &= \widehat{A}\,(Q - \mathbf{1}\,\mathrm{mean}_{v\in V}\, Q_v). \end{aligned} \tag{5}$$

Centering expresses a node's class association relative to the graph-wide average; neighborhood aggregation

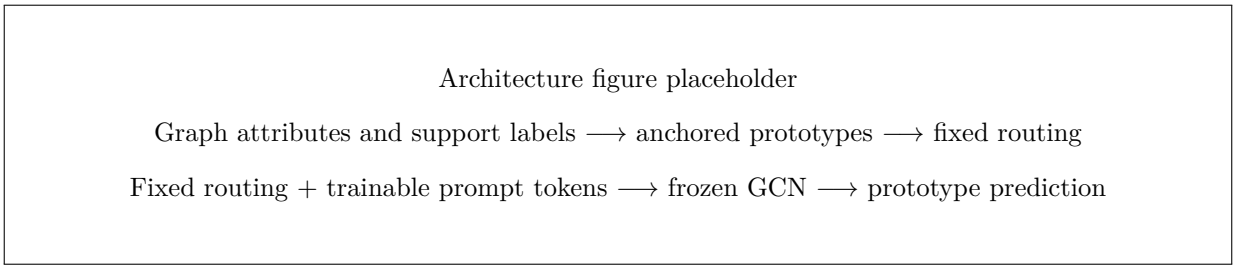


Figure 1: Overview of PRPrompt. Support labels initialize class references in aggregated attribute space. Unlabeled graph features refine these references, and the resulting node–prototype associations determine a fixed routing matrix. Only the class prompt tokens are optimized; the routed prompts enter the first hidden layer of a frozen GCN. Query labels are excluded from fitting and selection. The final architecture artwork is intentionally left as a placeholder.

incorporates adjacent association patterns. Q is a row-normalized association matrix, whereas B contains signed prompt-combination coefficients. We do not interpret B as a class-probability matrix. B is computed once per support set and kept fixed throughout training.

4.3 Adapting frozen hidden computations

Let H^{pre} be the first GCN layer's preactivation on the original graph. We learn a zero-initialized matrix $P \in \mathbb{R}^{C\times 128}$. With s the vector of channelwise sample standard deviations of H^{pre}, clamped below by 10^{-6}, the prompted hidden representation is

$$\widetilde{H}^{\text{pre}} = H^{\text{pre}} + 10\,[B\tanh(P)] \odot s. \tag{6}$$

The scale vector is broadcast over nodes. The existing LeakyReLU and second graph convolution then produce Z. Thus, prompt content is learned through support supervision, while routing and backbone parameters remain fixed. Zero initialization recovers the frozen encoder at the start of optimization. The trainable prompt counts are 896, 768, and 384 for Cora, CiteSeer, and PubMed.

4.4 Training and prediction

We optimize the prompt matrix for 100 Adam steps with learning rate 0.01 and no weight decay:

$$\begin{aligned}\mathcal{L}(P) = \frac{1}{|S|}\sum_{i\in S}\mathrm{CE}(\ell_i(P), y_i) \\ + 10^{-3}\,\mathrm{mean}(P\odot P).\end{aligned} \tag{7}$$

The support logits ℓ_i use cosine prototype classification. The same prototype readout is kept throughout the routing ablation. Support leave-one-out construction, prediction normalization, and temperatures are specified in Appendix A.

The routing is a one-time computation. For sparse graphs, the two attribute aggregation steps cost $O(|E|d)$; prototype refinement costs $O(nCd)$ per iteration; and multiplying B by prompt tokens costs $O(nC\cdot 128)$ per optimization step, in addition to the frozen encoder's forward and backward computations. Freezing weights reduces trainable parameters but does not eliminate gradient computation through the second convolution. We report parameter counts, not an unmeasured runtime advantage.

5 Experiments

5.1 Protocol and comparison methods

We evaluate Cora, CiteSeer, and PubMed at 1, 3, and 5 shots per class. The official training masks define the support pools and the official test masks define fixed query sets. Within a support draw, a shared per-class permutation yields nested 1-, 3-, and 5-shot sets. Each condition contains 20 support seeds and three training seeds. All methods share graph, checkpoint, and support/query identities within a condition. The pretrained source is the original edge-prediction checkpoint used by the project for each dataset; its pretraining seed is unavailable.

The main comparison contains: **Frozen**, with no prompt and the same final prototype readout as PRPrompt; **Full FT**, updating the pretrained encoder and a task readout; **GPF** and **GPF-plus**, adapting the input features; and **GraphPrompt**†, a diagonal readout-prompt implementation in the common node protocol. GPF-plus uses 20 prompt tokens. The dagger denotes a protocol adaptation, not a full reproduction of GraphPrompt's original subgraph-based pipeline. These prompt baselines are inspired by their original definitions [2, 9] but evaluated with the present backbone and selection rules.

PRPrompt and the shared-prompt ablation use one fixed recipe. For comparison methods that require selection, 3- and 5-shot fitting uses three support-only folds and four candidates: learning rate in $\{0.001, 0.01\}$ and weight decay in $\{0, 0.0001\}$. Each candidate fit and final fit uses 100 steps; 1-shot uses fixed parameters. Consequently, the methods do not have identical total optimization budgets. Appendix A gives the remaining settings.

Statistical unit. For a fixed condition, we first average three training seeds within each support draw, then report the mean and sample standard deviation over the 20 support draws. We never average performance across datasets or shot counts. Paired intervals resample support draws, preserving method pairing,

Table 1: Accuracy on few-shot node classification. Higher is better (↑).

Method	Cora			CiteSeer			PubMed		
	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot
Frozen	23.17 ± 5.48	29.42 ± 3.47	33.56 ± 5.03	21.14 ± 2.87	25.29 ± 2.16	27.38 ± 2.84	55.44 ± 11.42	65.55 ± 8.11	68.37 ± 3.41
Full FT	45.41 ± 10.18	61.54 ± 6.78	67.50 ± 5.06	35.76 ± 6.80	42.22 ± 5.88	47.89 ± 4.85	48.77 ± 8.54	59.75 ± 7.97	63.96 ± 5.18
GPF	15.20 ± 3.24	24.32 ± 4.66	28.17 ± 5.54	17.10 ± 2.64	20.36 ± 2.02	21.40 ± 2.57	36.29 ± 5.79	51.28 ± 6.93	55.83 ± 4.93
GPF-plus	30.80 ± 6.64	40.21 ± 4.80	53.14 ± 5.55	21.40 ± 4.60	28.92 ± 2.68	31.82 ± 2.60	41.12 ± 6.83	57.46 ± 4.96	61.37 ± 5.32
GraphPrompt†	21.88 ± 4.62	27.75 ± 3.64	31.36 ± 3.61	19.53 ± 3.34	24.83 ± 2.77	27.66 ± 2.49	50.24 ± 12.95	62.47 ± 5.54	66.94 ± 4.09
PRPrompt	**54.59** ± 9.18	**70.76** ± 2.76	**73.02** ± 1.95	**47.12** ± 6.31	**61.20** ± 4.98	**66.17** ± 2.16	**58.70** ± 9.78	**68.66** ± 4.58	**70.98** ± 2.46

Mean ± sample standard deviation (%). Within each fixed condition, three training seeds are averaged per support draw, then 20 support draws are summarized. **Bold**: best; underlined: second-best mean in each column. No averaging across columns. †: GraphPrompt under the common-protocol adaptation.

Table 2: Macro-F1 on few-shot node classification. Higher is better (↑).

Method	Cora			CiteSeer			PubMed		
	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot
Frozen	22.51 ± 4.82	29.07 ± 3.47	33.07 ± 4.47	20.39 ± 2.62	24.32 ± 1.87	26.45 ± 2.71	53.44 ± 10.49	63.19 ± 8.60	66.27 ± 3.55
Full FT	44.38 ± 10.07	60.94 ± 6.41	66.99 ± 4.52	30.99 ± 6.30	38.43 ± 5.80	44.62 ± 4.98	44.52 ± 9.85	57.64 ± 9.26	62.53 ± 6.05
GPF	7.08 ± 2.07	21.14 ± 6.64	25.87 ± 6.19	6.78 ± 1.56	13.81 ± 5.45	15.27 ± 5.85	25.62 ± 6.56	46.88 ± 9.09	52.42 ± 6.87
GPF-plus	27.08 ± 6.92	39.31 ± 4.55	51.82 ± 5.06	12.83 ± 4.10	26.16 ± 2.85	29.46 ± 2.64	34.89 ± 8.06	54.47 ± 4.51	58.79 ± 5.19
GraphPrompt†	21.03 ± 4.36	26.98 ± 3.41	30.82 ± 3.43	18.00 ± 2.95	23.71 ± 2.39	26.72 ± 2.49	45.35 ± 10.95	59.93 ± 5.76	64.79 ± 4.02
PRPrompt	**53.34** ± 8.94	**69.38** ± 2.91	**72.04** ± 1.77	**44.26** ± 5.75	**58.19** ± 4.67	**63.03** ± 2.12	**57.37** ± 10.07	**68.21** ± 4.08	**70.39** ± 2.47

Statistics and emphasis follow Table 1. Each column is a separate dataset/shot condition. †: GraphPrompt under the common-protocol adaptation.

with 10,000 bootstrap samples. They are unadjusted 95% intervals for the fixed graph and checkpoint. Accuracy and Macro-F1 are the primary reported metrics; the remaining metrics appear in Appendix B.

5.2 RQ1: Adaptation performance

This comparison tests whether the complete prototype-routed adaptation improves prediction under the same graph, checkpoint, and labeled support budget. Tables 1 and 2 report every condition, without a pooled average.

On Cora at 5 shots, PRPrompt reaches 73.02% accuracy compared with 33.56% for Frozen and 67.50% for Full FT. On CiteSeer at the same shot count, the corresponding accuracies are 66.17%, 27.38%, and 47.89%. PubMed has a stronger frozen reference: 68.37% at 5 shots, compared with 70.98% for PRPrompt and 63.96% for Full FT. Thus, the improvement over frozen pretraining is substantially smaller on PubMed, an important dependence that a cross-dataset average would hide.

Comparison with prompt operators. The same condition-wise comparison favors PRPrompt over each of the three prompt operators in Accuracy and Macro-F1. All reported Accuracy and Macro-F1 paired intervals against the five main references are positive. These intervals quantify seed-dependent variability within this setup; they do not establish superiority over all graph prompting methods or over independent graph populations. Full paired differences are provided in Appendix C.

5.3 RQ2: Value of prototype routing

The central ablation replaces B in Eq. (6) by the constant matrix with entries $1/C$. This retains the prompt matrix, scale, injection point, optimization, and final readout, but removes node-dependent allocation. The constant matrix is used directly; centering it would erase the prompt. We also retain the no-prompt Frozen reference.

Table 3 shows that a shared prompt alone does not account for the gains. At 5 shots, routing improves accuracy over the shared prompt by 36.94 percentage points on Cora, 40.45 on CiteSeer, and 3.02 on PubMed. The respective paired 95% intervals are $[34.87, 39.06]$, $[39.06, 41.84]$, and $[1.39, 4.51]$ percentage points. The improvement is also positive in both primary metrics in each 1- and 3-shot condition.

This comparison supports the complete prototype-routing construction relative to a shared prompt. It does not isolate prototype refinement, centering, and graph aggregation as separate causes. A uniform mixture also reduces the effective degrees of freedom despite retaining the nominal prompt parameter count. We therefore do not interpret the comparison as a capacity-matched proof of a unique routing mechanism.

Table 3: Prototype-routing ablation. Matched prompt recipe and readout.

Method	Cora			CiteSeer			PubMed		
	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot
Accuracy (%) ↑									
Frozen	23.17 ± 5.48	29.42 ± 3.47	33.56 ± 5.03	21.14 ± 2.87	25.29 ± 2.16	27.38 ± 2.84	55.44 ± 11.42	65.55 ± 8.11	68.37 ± 3.41
Shared prompt	22.62 ± 4.41	31.70 ± 4.95	36.08 ± 4.19	20.72 ± 2.46	24.01 ± 2.70	25.73 ± 2.78	53.84 ± 10.55	62.96 ± 7.94	67.95 ± 3.11
PRPrompt	**54.59** ± 9.18	**70.76** ± 2.76	**73.02** ± 1.95	**47.12** ± 6.31	**61.20** ± 4.98	**66.17** ± 2.16	**58.70** ± 9.78	**68.66** ± 4.58	**70.98** ± 2.46
Macro-F1 (%) ↑									
Frozen	22.51 ± 4.82	29.07 ± 3.47	33.07 ± 4.47	20.39 ± 2.62	24.32 ± 1.87	26.45 ± 2.71	53.44 ± 10.49	63.19 ± 8.60	66.27 ± 3.55
Shared prompt	21.65 ± 3.66	30.25 ± 4.85	34.89 ± 4.10	19.57 ± 2.33	22.67 ± 2.46	24.55 ± 2.74	51.93 ± 9.81	60.94 ± 8.60	66.29 ± 2.87
PRPrompt	**53.34** ± 8.94	**69.38** ± 2.91	**72.04** ± 1.77	**44.26** ± 5.75	**58.19** ± 4.67	**63.03** ± 2.12	**57.37** ± 10.07	**68.21** ± 4.08	**70.39** ± 2.47

Statistics and emphasis follow Table 1. Each column is a separate dataset/shot condition. Frozen: no prompt; shared prompt: uniform allocation; PRPrompt: prototype routing.

6 Limitations

The evaluation covers three citation graphs, one fixed pretrained checkpoint per graph, and same-graph known-class classification. Pretraining seeds are unknown, so variation across pretraining runs is not estimated. The reporting scope was chosen after the experiments were completed; the analysis is descriptive rather than a new independent confirmatory study. It cannot support claims about graph classification, unseen classes, or arbitrary pretraining sources.

The common-protocol prompt baselines and their restricted hyperparameter search need not attain their best published results. Baseline selection also incurs a different training budget from the fixed PRPrompt recipe. The routing ablation changes effective prompt capacity and does not distinguish every component of the routing construction. Finally, the method uses query features and graph-wide statistics; deployment without such transductive access requires separate evaluation.

7 Conclusion

PRPrompt uses support-anchored prototypes to organize the allocation of trainable hidden-layer prompts in a frozen graph encoder. In the evaluated Cora, CiteSeer, and PubMed settings, condition-wise comparisons and shared-prompt ablations support the value of this allocation. The evidence favors prototype routing as a practical adaptation design within the tested protocol, while leaving its generality across graph families and pretraining sources open.

Broader Impact

The study concerns public citation-network benchmarks and does not evaluate decisions about individuals. Errors can propagate through graph aggregation and unlabeled prototype refinement, particularly when the graph or class structure differs from the evaluated setting. Applications involving consequential decisions require domain-specific evaluation beyond these benchmarks.

References

[1] Q. Chen, L. Wang, B. Zheng, and G. Song. DAGPrompT: Pushing the Limits of Graph Prompting with a Distribution-aware Graph Prompt Tuning Approach. In *Proceedings of the ACM on Web Conference 2025*, WWW '25, pages 4346–4358. ACM, Apr. 2025. doi: 10.1145/3696410.3714917. URL http://dx.doi.org/10.1145/3696410.3714917.

[2] T. Fang, Y. Zhang, Y. Yang, C. Wang, and L. Chen. Universal Prompt Tuning for Graph Neural Networks. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 52464–52489. Curran Associates, Inc., 2023. doi: 10.52202/075280-2285. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/a4a1ee071ce0fe63b83bce507c9dc4d7-Paper-Conference.pdf.

[3] W. Hamilton, Z. Ying, and J. Leskovec. Inductive Representation Learning on Large Graphs. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/5dd9db5e033da9c6fb5ba83c7a7ebea9-Paper.pdf.

[4] Z. Hou, X. Liu, Y. Cen, Y. Dong, H. Yang, C. Wang, and J. Tang. GraphMAE: Self-Supervised Masked Graph Autoencoders. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '22, pages 594–604. ACM, Aug. 2022. doi: 10.1145/3534678.3539321. URL http://dx.doi.org/10.1145/3534678.3539321.

[5] W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. Pande, and J. Leskovec. Strategies for Pre-training Graph Neural Networks. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=HJlWWJSFDH.

[6] T. N. Kipf and M. Welling. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[7] J. Lee, W. Yang, and J. Kang. Subgraph-level universal prompt tuning. *Information Sciences*, 749:123516, Sept. 2026. ISSN 0020-0255. doi: 10.1016/j.ins.2026.123516. URL http://dx.doi.org/10.1016/j.ins.2026.123516.

[8] Z. Liang, L. Bai, and X. Yang. Attribute-guided Dynamic Prompt Learning for Graph Neural Networks. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(28):

23442–23450, Mar. 2026. ISSN 2159-5399. doi: 10.1609/aaai.v40i28.39515. URL http://dx.doi.org/10.1609/aaai.v40i28.39515.

[9] Z. Liu, X. Yu, Y. Fang, and X. Zhang. GraphPrompt: Unifying Pre-Training and Downstream Tasks for Graph Neural Networks. In *Proceedings of the ACM Web Conference 2023*, WWW '23, pages 417–428. ACM, Apr. 2023. doi: 10.1145/3543507.3583386. URL http://dx.doi.org/10.1145/3543507.3583386.

[10] Y. Ma, N. Yan, J. Li, M. Mortazavi, and N. V. Chawla. HetGPT: Harnessing the Power of Prompt Tuning in Pre-Trained Heterogeneous Graph Neural Networks. In *Proceedings of the ACM Web Conference 2024*, WWW '24, pages 1015–1023. ACM, May 2024. doi: 10.1145/3589334.3645685. URL http://dx.doi.org/10.1145/3589334.3645685.

[11] M. Ren, E. Triantafillou, S. Ravi, J. Snell, K. Swersky, J. B. Tenenbaum, H. Larochelle, and R. S. Zemel. Meta-Learning for Semi-Supervised Few-Shot Classification, 2018. URL https://arxiv.org/abs/1803.00676.

[12] J. Snell, K. Swersky, and R. Zemel. Prototypical Networks for Few-shot Learning. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/cb8da6767461f2812ae4290eac7cbc42-Paper.pdf.

[13] M. Sun, K. Zhou, X. He, Y. Wang, and X. Wang. GPPT: Graph Pre-training and Prompt Tuning to Generalize Graph Neural Networks. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '22, pages 1717–1727. ACM, Aug. 2022. doi: 10.1145/3534678.3539249. URL http://dx.doi.org/10.1145/3534678.3539249.

[14] X. Sun, H. Cheng, J. Li, B. Liu, and J. Guan. All in One: Multi-Task Prompting for Graph Neural Networks. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '23, pages 2120–2131. ACM, Aug. 2023. doi: 10.1145/3580305.3599256. URL http://dx.doi.org/10.1145/3580305.3599256.

[15] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. Graph Attention Networks. *International Conference on Learning Representations*, 2018. URL https://openreview.net/forum?id=rJXMpikCZ. accepted as poster.

[16] P. Veličković, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, and R. D. Hjelm. Deep Graph Infomax. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=rklz9iAcKQ.

[17] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen. Graph Contrastive Learning with Augmentations. In H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 5812–5823. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/3fe230348e9a12c13120749e3f9fa4cd-Paper.pdf.

[18] X. Yu, Y. Fang, Z. Liu, and X. Zhang. HGPrompt: Bridging Homogeneous and Heterogeneous Graphs for Few-Shot Prompt Learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(15):16578–16586, Mar. 2024. ISSN 2159-5399. doi: 10.1609/aaai.v38i15.29596. URL http://dx.doi.org/10.1609/aaai.v38i15.29596.

[19] X. Yu, C. Zhou, Y. Fang, and X. Zhang. MultiGPrompt for Multi-Task Pre-Training and Prompting on Graphs. In *Proceedings of the ACM Web Conference 2024*, WWW '24, pages 515–526. ACM, May 2024. doi: 10.1145/3589334.3645423. URL http://dx.doi.org/10.1145/3589334.3645423.

[20] X. Yu, J. Zhang, Y. Fang, and R. Jiang. Non-Homophilic Graph Pre-Training and Prompt Learning. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*, KDD '25, pages 1844–1854. ACM, July 2025. doi: 10.1145/3690624.3709219. URL http://dx.doi.org/10.1145/3690624.3709219.

[21] J. Zhu, Z. Ding, J. Yu, J. Tan, X. Li, and W. Qian. RELIEF: Reinforcement Learning Empowered Graph Feature Prompt Tuning. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*, KDD '25, pages 2159–2170. ACM, July 2025. doi: 10.1145/3690624.3709252. URL http://dx.doi.org/10.1145/3690624.3709252.

[22] C. Zi, H. Zhao, X. Sun, Y. Lin, H. Cheng, and J. Li. ProG: A Graph Prompt Learning Benchmark. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 95406–95437. Curran Associates, Inc., 2024. doi: 10.52202/079017-3023. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/ad3e803a977f4279330c6ab7245937c6-Paper-Datasets_and_Benchmarks_Track.pdf.

A Reproducibility

A.1 Graph preprocessing and fixed checkpoints

Table 4 records the dataset dimensions from the execution plan. Features use the dataset loader's row normalization. Support pools and query sets come from the official training and test masks; validation labels are not used for fitting or selection. All other nodes remain visible as unlabeled graph context. Each task contains all classes of its graph, with the requested number of support nodes per class.

Table 4: Dataset and prompt sizes.

Dataset	Nodes	Features	Classes	Prompt params.
Cora	2,708	1,433	7	896
CiteSeer	3,327	3,703	6	768
PubMed	19,717	500	3	384

The encoder consists of two GCNConv layers of width 128 with LeakyReLU between them. The first layer includes the pretrained affine transformation and graph aggregation; the prompt is added to its output before the nonlinearity. The second convolution has no additional activation in the evaluated encoder. The same checkpoint is used by every comparison method on a dataset. Checkpoint, graph, and split hashes are preserved in the accompanying result records. The recorded pretraining family is `Edgepred_Gprompt`; the pretraining random seed is unavailable.

A.2 Training and prediction details

The main text describes the active forward computation. For reproducibility, Adam uses $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, learning rate 0.01, and weight decay zero. The prompt regularizer is the mean of squared token entries multiplied by 10^{-3}. It is separate from optimizer weight decay. The channel scale uses the sample standard deviation, with denominator $n-1$. Euclidean normalization uses the implementation's 10^{-12} norm floor. No distillation or query-label objective is active.

For the support loss, let $\bar{z}_i = \mathrm{norm}(Z_i)$ and $m_c = \sum_{j \in S_c} \bar{z}_j$. At $k > 1$, the normalized prototype used for training support node i is

$$p_c^{(-i)} = \mathrm{norm}\left(\frac{m_c - \mathbb{I}[y_i = c]\bar{z}_i}{k - \mathbb{I}[y_i = c]} \right), \qquad \ell_{ic} = \bar{z}_i^\top p_c^{(-i)}/0.2. \tag{8}$$

For $k = 1$, the inclusive class prototype $\mathrm{norm}(m_c/k)$ replaces $p_c^{(-i)}$. During prediction, define

$$z_i^* = \mathrm{norm}\left(Z_i - \frac{1}{n} \sum_{j \in V} Z_j \right), \quad p_c^* = \mathrm{norm}\left(\frac{1}{k} \sum_{i \in S_c} z_i^* \right), \quad \widehat{y}_i = \arg\max_c (z_i^*)^\top p_c^*. \tag{9}$$

The probability scores for all five metrics are obtained by applying softmax to cosine logits divided by 0.2. Training does not apply the output centering used at prediction. The readout is a fixed implementation choice, not an experimental factor in this paper.

Procedure. Given a graph, a fixed checkpoint, and support labels:

1. Normalize adjacency and compute Eq. (1) and the first-layer preactivation.
2. Initialize support prototypes with Eq. (2) and apply Eqs. (3)–(4) three times.
3. Compute the fixed route with Eq. (5); for the shared-prompt ablation, replace it by $B_{ic} = 1/C$.
4. Initialize $P = 0$ and cache the hidden-channel scales. Keep the encoder and route fixed.
5. For 100 steps, run the prompted encoder, form support logits, and update P using Eq. (7).
6. Compute final prototype scores using Eq. (9); seal predictions before scoring with query labels.

A.3 Baseline fitting and seed control

The support-draw seeds are 30093000 through 30093019 and training seeds are 0, 1, and 2. A draw uses a seeded permutation in each class's support pool, taking its first k entries. All methods receive the same support identities for each draw. The no-training Frozen method retains the same three seed slots for alignment, but those slots do not represent three independent fits.

The learnable comparison methods use support cross-entropy. Full FT and the input-prompt methods include a task readout, whereas the GraphPrompt adaptation uses its prompted representation with prototype prediction. GPF adds a shared feature vector; GPF-plus learns 20 feature prompts and an input-dependent mixture. Their final one-shot parameters are learning rate 0.01 and weight decay 0.0001. For 3 and 5 shots, each of three support-only folds holds out one node per class. Four candidate configurations are ranked by mean held-out support cross-entropy, then the selected method is refit on the complete support set. No external validation-label budget is used. This protocol controls labeled information and final step count, but not total selection compute or readout parameterization.

A.4 Condition-specific statistics

For a fixed dataset, shot count, checkpoint, and method, let a_{st} denote a metric with support seed s and training seed t. We compute

$$\bar{a}_s = \frac{1}{3} \sum_{t=1}^{3} a_{st}, \qquad \widehat{a} = \frac{1}{20} \sum_{s=1}^{20} \bar{a}_s, \qquad \widehat{\sigma} = \sqrt{\frac{1}{19} \sum_{s=1}^{20} (\bar{a}_s - \widehat{a})^2}. \tag{10}$$

Tables report $100\widehat{a} \pm 100\widehat{\sigma}$, not the standard error. For paired comparisons, $d_s = \bar{a}_s^{\mathrm{PRPrompt}} - \bar{a}_s^{\mathrm{reference}}$ is resampled as a support-draw block. The percentile bounds of 10,000 resampled means form the reported 95% interval. Existing intervals are reused without modification. No metric average or mean effect is computed across different datasets, shot counts, or methods.

Accuracy, Macro-F1, Micro-F1, macro one-vs-rest ROC-AUC, and macro average precision are retained in the data package. In this single-label multiclass setting, Micro-F1 coincides with Accuracy; it is included for completeness. The fixed query set is shared across support draws, so intervals concern support selection and training variation conditional on that graph, not independent samples of graphs.

A.5 Environment, provenance, and numerical qualification

The execution plan records Python 3.11.16, PyTorch 2.11.0+cu130, and CUDA 13.0 in the reused ProG environment. Per-unit records identify CUDA devices and fitting durations. An isolated total GPU-hour budget for the present reporting scope and a verified accelerator inventory are not provided by the extracted package; no efficiency comparison is inferred from these results.

The tables derive from the sealed three-dataset package. It contains 3,240 main-comparison units and 540 additional shared-prompt units; PRPrompt contributes 540 units. Source-metric checksums and paired graph/checkpoint/support/query identities were verified during extraction. The source numerical audits report exact readout reproduction from saved representations for all 3,780 selected units, passing gate/forward tolerances, and no changed query classes in fresh forwards. These are archived verification results, not newly rerun experiments, and passing a tolerance does not imply bitwise-identical CUDA arithmetic.

Source code and the table data are available in the accompanying repository at https://github.com/Mes-sala/PRPrompt. The source revision of the recorded experiments is `0328a2bb68f249a34c89557178b6233daf5ce20d`. The paper package includes table-generation and static-validation scripts, hashes, and bibliographic provenance. Model checkpoints and optimizer states remain on the experiment server and are excluded from the local paper package. This manuscript reports existing experiments; no new training or bootstrap analysis was performed for its preparation.

B Additional Metrics

The following tables extend the main comparison with the remaining stored metrics under the same seed aggregation rule. Every column is a separate dataset/shot condition.

Table 5: Micro-F1 on few-shot node classification. Higher is better (↑).

Method	Cora			CiteSeer			PubMed		
	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot
Frozen	23.17 ± 5.48	29.42 ± 3.47	33.56 ± 5.03	21.14 ± 2.87	25.29 ± 2.16	27.38 ± 2.84	55.44 ± 11.42	65.55 ± 8.11	68.37 ± 3.41
Full FT	45.41 ± 10.18	61.54 ± 6.78	67.50 ± 5.06	35.76 ± 6.80	42.22 ± 5.88	47.89 ± 4.85	48.77 ± 8.54	59.75 ± 7.97	63.96 ± 5.18
GPF	15.20 ± 3.24	24.32 ± 4.66	28.17 ± 5.54	17.10 ± 2.64	20.36 ± 2.02	21.40 ± 2.57	36.29 ± 5.79	51.28 ± 6.93	55.83 ± 4.93
GPF-plus	30.80 ± 6.64	40.21 ± 4.80	53.14 ± 5.55	21.40 ± 4.60	28.92 ± 2.68	31.82 ± 2.60	41.12 ± 6.83	57.46 ± 4.96	61.37 ± 5.32
GraphPrompt†	21.88 ± 4.62	27.75 ± 3.64	31.36 ± 3.61	19.53 ± 3.34	24.83 ± 2.77	27.66 ± 2.49	50.24 ± 12.95	62.47 ± 5.54	66.94 ± 4.09
PRPrompt	**54.59** ± 9.18	**70.76** ± 2.76	**73.02** ± 1.95	**47.12** ± 6.31	**61.20** ± 4.98	**66.17** ± 2.16	**58.70** ± 9.78	**68.66** ± 4.58	**70.98** ± 2.46

Statistics and emphasis follow Table 1. Each column is a separate dataset/shot condition.

Table 6: Macro ROC-AUC on few-shot node classification. Higher is better (↑).

Method	Cora			CiteSeer			PubMed		
	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot
Frozen	62.95 ± 4.93	68.95 ± 2.99	72.36 ± 2.85	56.11 ± 3.24	60.17 ± 2.06	62.69 ± 2.35	73.87 ± 11.03	82.17 ± 6.59	84.83 ± 2.59
Full FT	81.83 ± 5.73	89.94 ± 2.35	**92.58** ± 1.26	70.97 ± 4.63	76.04 ± 3.00	79.54 ± 2.22	70.73 ± 9.76	81.00 ± 3.74	84.31 ± 2.45
GPF	53.20 ± 2.50	64.61 ± 4.25	68.65 ± 4.34	53.08 ± 1.84	55.85 ± 1.91	57.93 ± 1.77	60.13 ± 8.31	72.19 ± 4.91	74.64 ± 4.10
GPF-plus	74.73 ± 4.00	78.13 ± 2.49	85.66 ± 2.71	64.19 ± 4.07	65.37 ± 3.23	68.91 ± 2.38	68.63 ± 6.61	76.82 ± 4.24	79.98 ± 2.95
GraphPrompt†	61.93 ± 4.85	68.05 ± 3.01	71.01 ± 2.66	54.39 ± 4.21	59.99 ± 2.60	62.67 ± 2.33	70.43 ± 11.06	81.13 ± 4.42	83.57 ± 2.81
PRPrompt	**83.06** ± 5.42	**90.84** ± 1.97	92.47 ± 1.11	**74.71** ± 5.81	**82.82** ± 3.08	**85.86** ± 1.52	**76.48** ± 10.43	**86.29** ± 3.10	**87.03** ± 1.87

Statistics and emphasis follow Table 1. Each column is a separate dataset/shot condition.

Table 7: Macro AP on few-shot node classification. Higher is better (↑).

Method	Cora			CiteSeer			PubMed		
	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot
Frozen	23.64 ± 4.26	29.20 ± 3.71	33.16 ± 4.26	21.53 ± 2.32	24.17 ± 1.59	26.46 ± 2.07	57.52 ± 10.93	67.27 ± 8.59	70.34 ± 4.13
Full FT	54.01 ± 10.10	70.20 ± 4.98	75.99 ± 3.16	38.74 ± 5.84	45.56 ± 4.79	51.03 ± 4.40	55.36 ± 10.55	67.52 ± 6.82	72.33 ± 5.00
GPF	16.64 ± 1.47	24.89 ± 3.11	28.68 ± 3.71	19.66 ± 1.43	21.40 ± 1.15	22.88 ± 1.45	43.93 ± 6.89	55.29 ± 5.93	58.42 ± 5.12
GPF-plus	40.37 ± 5.88	44.43 ± 4.09	58.84 ± 6.14	29.38 ± 4.20	30.59 ± 3.23	34.40 ± 2.70	52.61 ± 8.08	60.88 ± 5.58	65.42 ± 4.92
GraphPrompt†	23.15 ± 3.89	27.06 ± 2.89	29.92 ± 3.11	20.61 ± 2.27	23.97 ± 1.89	26.04 ± 1.65	52.39 ± 9.87	65.03 ± 5.79	68.42 ± 3.67
PRPrompt	**59.04** ± 9.07	**74.33** ± 3.38	**77.64** ± 1.87	**45.99** ± 6.79	**59.15** ± 4.80	**64.02** ± 2.66	**61.58** ± 11.39	**74.22** ± 5.19	**75.35** ± 3.34

Statistics and emphasis follow Table 1. Each column is a separate dataset/shot condition.

C Paired Differences and Uncertainty

Each cell reports PRPrompt minus the named reference in percentage points, with its unadjusted 95% paired support-bootstrap interval. All entries preserve the original paired statistics. Tables group conditions for readability but do not average them.

Table 8: Paired Accuracy gains of PRPrompt. Differences in percentage points.

Reference	Cora			CiteSeer			PubMed		
	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot
Frozen	+31.42 [28.20, 34.59]	+41.34 [39.63, 43.04]	+39.46 [37.23, 41.61]	+25.98 [23.72, 28.21]	+35.91 [33.05, 38.39]	+38.78 [37.37, 40.18]	+3.26 [0.40, 6.22]	+3.11 [0.38, 6.27]	+2.61 [1.13, 4.22]
Full FT	+9.17 [6.58, 11.87]	+9.22 [6.75, 11.79]	+5.52 [3.73, 7.50]	+11.36 [9.57, 13.18]	+18.98 [16.78, 20.94]	+18.28 [16.22, 20.13]	+9.93 [6.77, 13.14]	+8.90 [5.89, 12.26]	+7.02 [4.96, 9.14]
GPF	+39.38 [35.60, 42.98]	+46.44 [44.21, 48.74]	+44.86 [42.52, 47.57]	+30.02 [27.04, 33.00]	+40.83 [38.11, 43.06]	+44.77 [43.32, 46.16]	+22.40 [18.82, 26.24]	+17.37 [14.16, 21.02]	+15.15 [13.30, 17.01]
GPF-plus	+23.78 [19.96, 27.47]	+30.55 [28.29, 32.52]	+19.88 [17.62, 22.16]	+25.71 [22.89, 28.45]	+32.27 [30.45, 34.03]	+34.35 [33.05, 35.65]	+17.57 [13.28, 21.86]	+11.20 [9.47, 13.11]	+9.61 [7.84, 11.49]
GraphPrompt†	+32.70 [29.19, 36.29]	+43.01 [41.38, 44.73]	+41.66 [40.03, 43.20]	+27.59 [24.97, 30.09]	+36.37 [33.26, 38.82]	+38.51 [37.14, 39.90]	+8.45 [3.60, 13.47]	+6.19 [4.04, 8.30]	+4.03 [1.94, 6.36]
Shared prompt	+31.96 [28.59, 35.21]	+39.06 [36.49, 41.44]	+36.94 [34.87, 39.06]	+26.40 [24.07, 28.73]	+37.18 [34.90, 39.33]	+40.45 [39.06, 41.84]	+4.85 [1.26, 8.26]	+5.69 [3.25, 8.27]	+3.02 [1.39, 4.51]

Each cell: PRPrompt minus the reference (top); unadjusted 95% paired support-bootstrap interval (below). All values preserve the original condition-wise statistics.

Table 9: Paired Macro-F1 gains of PRPrompt. Differences in percentage points.

Reference	Cora			CiteSeer			PubMed		
	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot	1-shot	3-shot	5-shot
Frozen	+30.84 [27.77, 33.84]	+40.32 [38.64, 42.08]	+38.97 [36.88, 40.99]	+23.87 [21.80, 25.77]	+33.87 [31.34, 36.14]	+36.58 [35.22, 37.95]	+3.93 [0.86, 7.00]	+5.02 [2.27, 8.44]	+4.12 [2.53, 5.91]
Full FT	+8.97 [6.40, 11.77]	+8.44 [6.12, 10.90]	+5.05 [3.34, 6.86]	+13.27 [11.17, 15.31]	+19.76 [17.45, 22.00]	+18.42 [16.07, 20.60]	+12.86 [9.39, 16.37]	+10.57 [7.08, 14.41]	+7.86 [5.54, 10.28]
GPF	+46.27 [42.36, 49.82]	+48.25 [45.22, 51.53]	+46.17 [43.71, 49.32]	+37.48 [34.93, 39.92]	+44.38 [40.82, 47.90]	+47.77 [44.95, 50.72]	+31.76 [28.23, 35.27]	+21.33 [17.31, 25.89]	+17.97 [15.46, 20.67]
GPF-plus	+26.26 [22.23, 30.20]	+30.07 [28.04, 31.97]	+20.22 [18.02, 22.39]	+31.43 [28.61, 34.12]	+32.03 [30.10, 33.81]	+33.57 [32.22, 34.87]	+22.48 [18.10, 26.98]	+13.74 [12.07, 15.56]	+11.61 [9.80, 13.53]
GraphPrompt†	+32.32 [28.93, 35.68]	+42.41 [40.61, 44.20]	+41.22 [39.64, 42.75]	+26.26 [23.90, 28.62]	+34.48 [31.80, 36.73]	+36.31 [34.99, 37.65]	+12.02 [7.33, 16.52]	+8.27 [6.21, 10.36]	+5.60 [3.49, 7.88]
Shared prompt	+31.70 [28.29, 35.00]	+39.13 [36.60, 41.48]	+37.15 [35.09, 39.23]	+24.69 [22.70, 26.65]	+35.52 [33.31, 37.55]	+38.48 [37.22, 39.72]	+5.44 [1.84, 9.07]	+7.27 [4.64, 10.29]	+4.10 [2.56, 5.58]

Each cell: PRPrompt minus the reference (top); unadjusted 95% paired support-bootstrap interval (below). All values preserve the original condition-wise statistics.